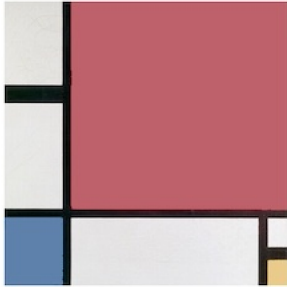


# Visual Scene Graph

## Baseline Experiment



Use a binding operation to encode the colored features in this image, and to bind those three features to the overall scene. A memory query either completes the scene (replacement update rule) or predicts the missing elements (difference update rule).

## Requirements

```
<< "Circuits.m"
```

## Circuit

```
{  
  "$schema": "https://creatingintelligence.org/schemas/circuit-v1.json",  
  "hyperparameters": {"default": [2000, 10]},  
}
```

```

"dataflow": [
  {"component": "input", "plugin": "codec", "send": [11]},
  {"component": "auto", "send": [21], "receive": [11], "learn": true, "decimate": 0.95},
  {"component": "output", "plugin": "codec", "receive": [21]}
]
}

```

```

circ = Circuit[ json];
f = circ["function"];
circ["schematics"][ImageSize→ 220]

```



## Train

```

f[ {"01", "large", "red", "top", "right"}];
f[ {"02", "medium", "blue", "bottom", "left"}];
f[ {"03", "small", "yellow", "bottom", "right"}];
f[ {"Scene", "01", "02", "03"}];

```

## Test

```
f@ "Scene"
```

```
Sequence[{01, 02, 03, Scene}]
```

```
f@ "01"
```

```
Sequence[{large, 01, 02, 03, red, right, Scene, top}]
```

```
f@ "red"
```

```
Sequence[{large, 01, red, right, top}]
```

**f@ "top"**

Sequence[{{large, 01, red, right, top}}]

**f@ "bottom"**

Sequence[{{blue, bottom, left, medium, 02, 03, right, small, yellow}}]

**f@ {"bottom", "right"}**

Sequence[{{bottom, 03, right, small, yellow}}]

**f@ {"blue", "medium"}**

Sequence[{{blue, bottom, left, medium, 02}}]

**f@ "left"**

Sequence[{{blue, bottom, left, medium, 02}}]

**f@ "right"**

Sequence[{{bottom, large, 01, 03, red, right, small, top, yellow}}]

**f@ "small"**

Sequence[{{bottom, 03, right, small, yellow}}]

**f@ {"Scene", "blue"}**

Sequence[{{blue, Scene}}]